

## Class introduction

Ryan Giordano



# Key concepts

This class will focus almost entirely on *supervised learning*.

Here are the key concepts:

- ▶ What does it mean to do well on a supervised learning problem?
  - ▶ Random data, loss, and risk
  - ▶ **Empirical risk minimization, generalization**
- ▶ What would we do if we had an infinite amount of data?
  - ▶ Generating computational tools to fit arbitrary functions
  - ▶ **Expressive function classes**
- ▶ What goes wrong when we have only a limited amount of data?
  - ▶ Avoid overfitting by limiting the expressivity of function classes
  - ▶ **Regularization**
- ▶ How do we find a sweet spot between expressiveness and overfitting?
  - ▶ Theory: **generalization bounds**
  - ▶ Practice: **out-of-sample error**

# Day one walkthrough

- ▶ Fitting a given function if all you can do is query it
  - ▶ Function classes
    - ▶ What if you need to make a prediction at a point you've never seen before?
    - ▶ An approximation scheme comes with a class of functions that it can represent.
  - ▶ Loss
    - ▶ You've chosen some approximation. How well does it do?
    - ▶ When you observe the function exactly, is there any reason not to choose an arbitrarily expressive function class?
    - ▶ Plot best function in class versus truth as function class increases
  - ▶ Randomly chosen query locations
    - ▶ Where should you query?
    - ▶ You need to know that future queries will need to look like the present queries somehow
  - ▶ Risk (versus pointwise loss)
    - ▶ Can you guaranteed low loss on a point you've never seen before? (Spiky functions)
    - ▶ Assumptions about confounders

# Day two walkthrough

- ▶ Random responses
  - ▶ Conditional distributions of responses
    - ▶ What happens if you query the same point twice and get different answers? Conditional distribution
  - ▶ Irreducible loss
    - ▶ What's the best you can do if there's noise in the response?
    - ▶ Optimal prediction function, and gap with function class
  - ▶ Overfitting
    - ▶ Suppose you have chosen a nice, expressive class, but have random  $y$ .
    - ▶ Can you tell whether you've overfit just by looking at the data? (No)
    - ▶ Plot best function in class versus truth vs data
  - ▶ Checking for overfitting for a single  $x$ 
    - ▶ For a given point, ideally check the same point multiple times
    - ▶ For a given point, check nearby points and assume smoothness
  - ▶ Checking for overfitting when risk is the goal
    - ▶ If all we want is risk, get a new point to check.
    - ▶ And if your data is IID and you can't get a new point, just leave a point out
    - ▶ If the function class is smooth, and you can "borrow" from neighbors, you might hope that the estimate goes to the best function in the class

## QSAR Fish Toxicity Dataset

These examples will be based on the UCI QSAR Fish Toxicity Dataset.

The dataset consists of a set of 908 chemicals (one row each), with data for:

- ▶ LC50: the concentration that causes death in 50% of test fish over a test duration of 96 hours.
- ▶ MLOGP: (molecular properties)
- ▶ CIC0: (information indices)
- ▶ GATS1i: (2D autocorrelations)
- ▶ NdsC: (atom-type counts)
- ▶ NdsCH: (atom-type counts)
- ▶ SM1\_Dz: (2D matrix-based descriptors)



Figure 1: Sad fish

“Quantitative structure–activity relationship” (QSAR) models predict some chemical property (toxicity to the fathead minnow) from structural features of the molecules.

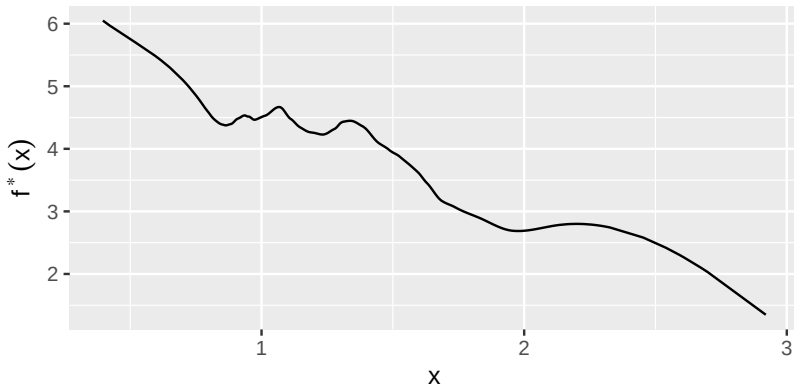
**Task:** Set  $x = \text{GATS1i}$ ,  $y = \text{LC50}$ , and use ML to predict fish mortality from 2D autocorrelations.

### **i** Question

Describe some concrete settings when you would want to use an ML model that could perform this task.

**Note:** I will be simulating data using a “ground truth” that is loosely based on patterns from the actual data.

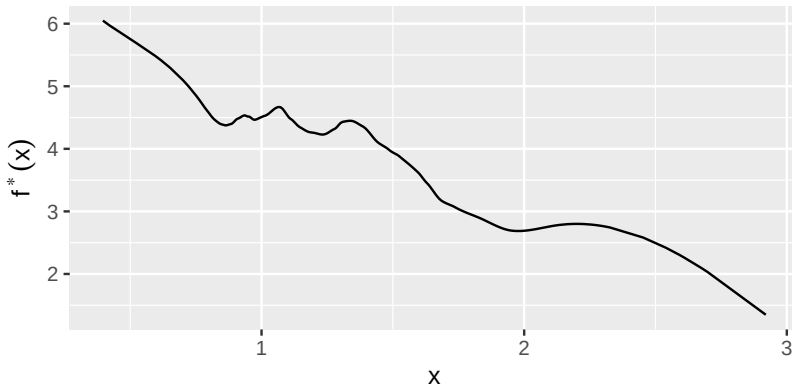
## Fitting a function



Let's imagine that  $y$  (fish toxicity) is determined exactly by  $x$  (2D autocorrelations).

That means  $y = f^*(x)$  for some  $f^*$ .

## Fitting a function



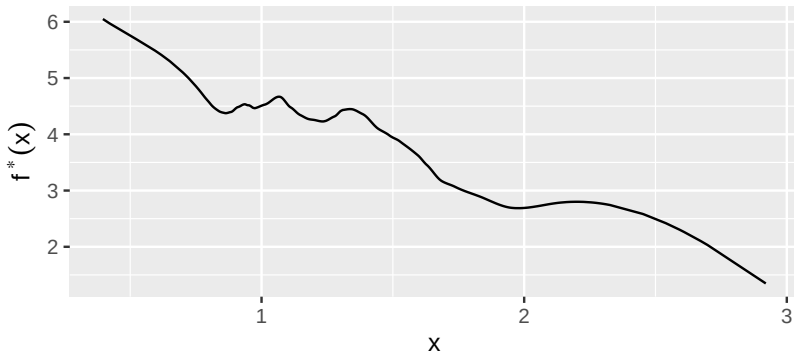
Let's imagine that  $y$  (fish toxicity) is determined exactly by  $x$  (2D autocorrelations).

That means  $y = f^*(x)$  for some  $f^*$ .

We don't know  $f^*$ , but we can *query* it by selecting  $x$  (a molecule) and measuring  $y$  (seeing what concentration kills minnows)



## Fitting a function



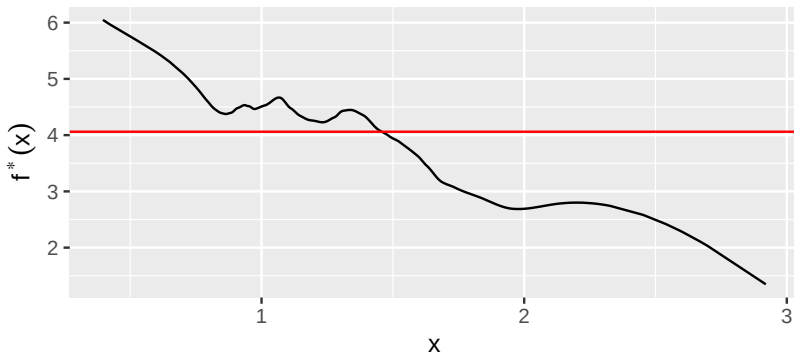
Task: Query  $(x_n, y_n)$  pairs and make a *program* that returns  $\hat{f}(x) \approx f^*(x)$ .

### Question

Suppose you can query a very large number of datapoints.

- ▶ Can you ever perfectly approximate  $f^*(x_{\text{new}})$  at an unseen  $x_{\text{new}}$ ?
- ▶ Propose the simplest procedure you can think of (even if not very good).
- ▶ Propose a more complex procedure that will do a better job.

## Constant functions

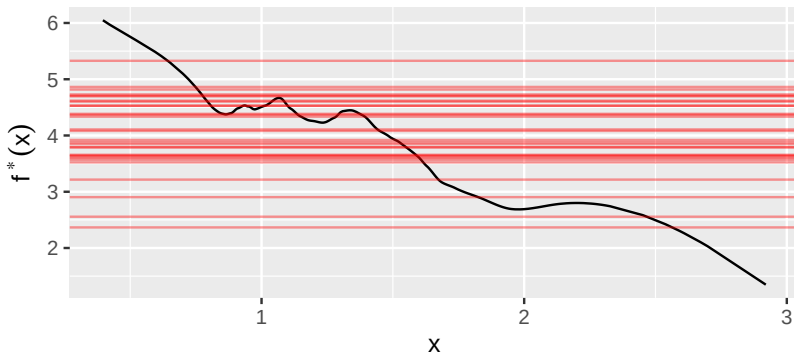


The simplest approximation is arguably the constant  $\hat{f}(x) = \frac{1}{N} \sum_{n=1}^N f^*(x_n)$ .

### Question

- ▶ Think of an application for which a good constant is enough.
- ▶ Think of an application for which a good constant is not enough.
- ▶ Will the constant you estimate depend on which  $x_n$  you select? How?
- ▶ Can you think of other ways to take the data and choose the constant?

## Constant functions



Using a constant function selects  $\hat{f}$  from the *function class*:

$$\mathcal{F}_{constant} := \{f : f(x) = c \text{ for some real-valued } c\}$$

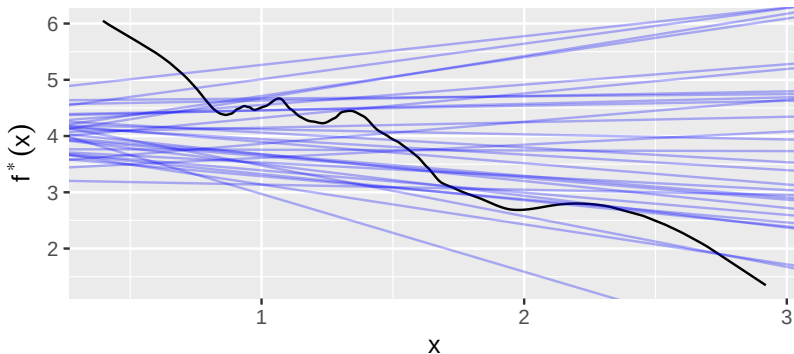
The function class is the set of *all functions that you could represent* with your procedure.

An “estimation procedure” takes in data  $\mathcal{D} = \{(x_1, y_1), \dots, (x_N, y_N)\}$  and returns a function  $\hat{f} \in \mathcal{F}_{constant}$ .

This is equivalent to choosing a  $\hat{c}$  and taking  $\hat{f}(x) = \hat{c}$ .

This class is about designing and studying estimation procedures.

## Linear functions



We can also define the class of linear functions:

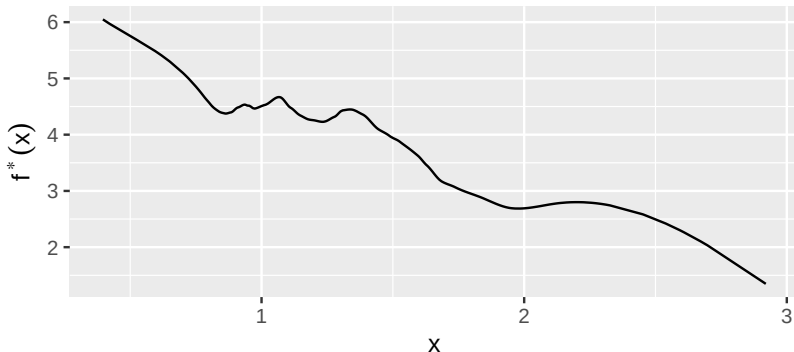
$$\mathcal{F}_{constant} := \{f : f(x) = c \text{ for some real-valued } c\}$$

$$\mathcal{F}_{linear} := \{f : f(x) = c + bx \text{ for some real-valued } c, b\}$$

Every constant function is also a linear function, but some linear functions are not constant functions.

We say that  $\mathcal{F}_{linear}$  is a *more expressive function class* than  $\mathcal{F}_{constant}$ .

## Function classes

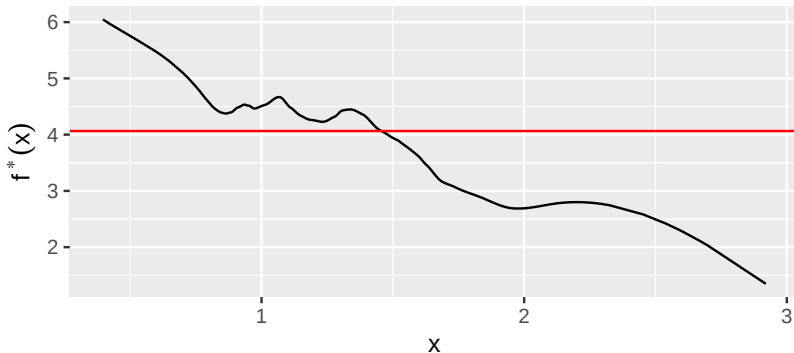


**A key theme of this class is developing expressive function classes!**

### **i** Question

Identify the function classes for some of the examples given earlier. Which are more expressive? Which are the least expressive?

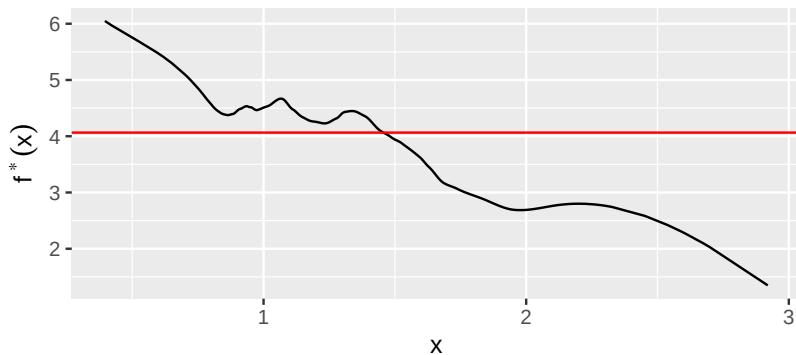
## Loss



We want to choose  $\hat{f} \in \mathcal{F}$  that is a “good approximation,” so we have to say what “good” means.

Suppose we have some  $x$  with associated  $y$ . Let the “loss function”  $\mathcal{L}(f(x), y)$  measure how “bad it is” to predict  $f(x)$  when the truth was  $y$ .

## Loss



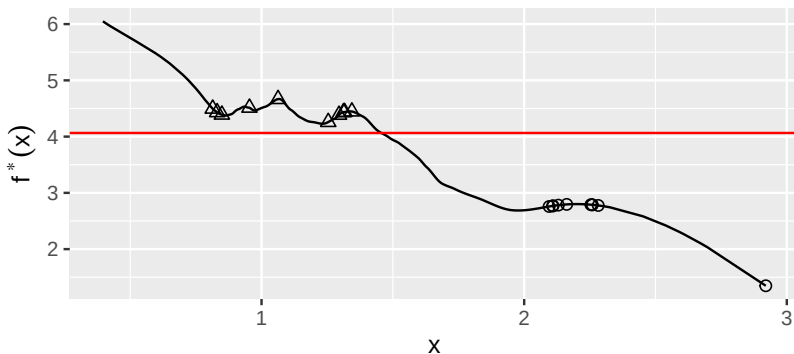
Common examples:

- ▶  $\mathcal{L}(f(x), y) = (f(x) - y)^2$
- ▶  $\mathcal{L}(f(x), y) = |f(x) - y|$
- ▶  $\mathcal{L}(f(x), y) = I(f(x) \neq y)$  (Here,  $I(\cdot)$  is the “indicator function.” It evaluates to 1 if its argument is true, and 0 otherwise.)

### Question

What are some properties any loss function should satisfy?

## Risk



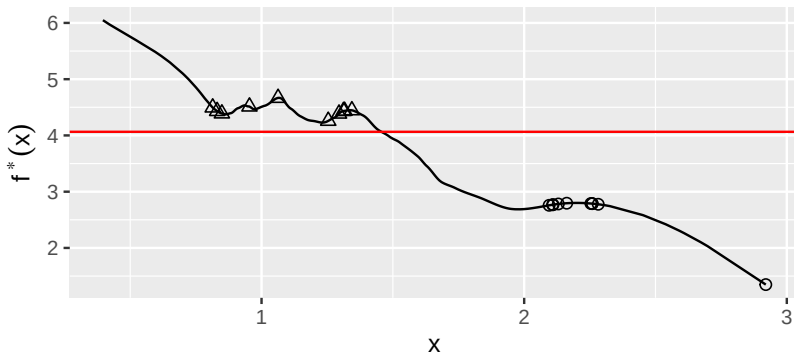
The loss is defined at a single datapoint. It makes sense to *average* the loss over multiple datapoints:

$$\frac{1}{N} \sum_{n=1}^N \mathcal{L}(f(x_n), y_n)$$

Note that, for a given  $f$ , the average loss will be different depending where the points are! **Where should you evaluate your loss?**



## Risk



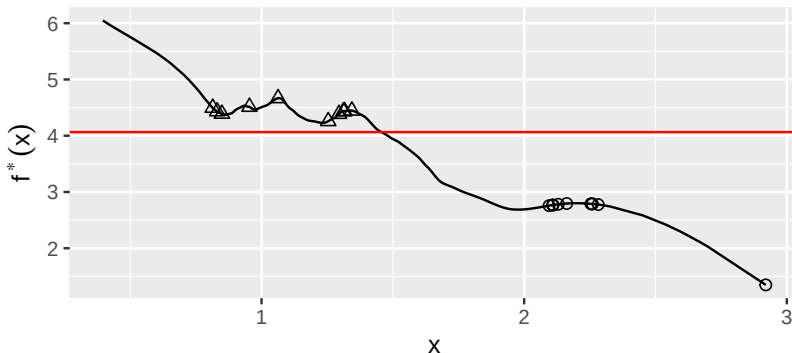
If we believe that future  $x$  will be drawn randomly, independently, and identically distributed (IID) from a future distribution  $p(x)$ , then it makes sense to measure the *average loss*.

This average loss is called “risk”.

$$\mathcal{R}(f) := \mathbb{E}_{p(x,y)} [\mathcal{L}(f(x), y)] \quad \text{”Risk”}$$

Note that risk depends only on the function — the distribution or data is considered fixed.

## Risk



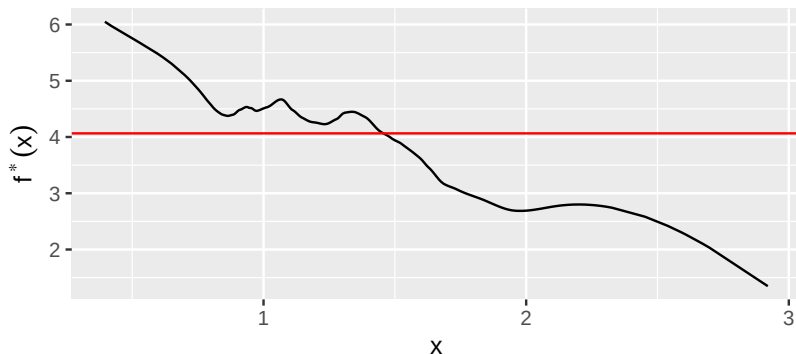
If we want to learn a function that makes  $\mathcal{R}(f)$  small, we should draw training data  $x_n \stackrel{\text{iid}}{\sim} p(x)$  and form the “empirical risk:”

$$\mathcal{R}(f) := \mathbb{E}_{p(x,y)} [\mathcal{L}(f(x), y)] \quad \text{”Risk”}$$

$$\hat{\mathcal{R}}(f) := \frac{1}{N} \sum_{n=1}^N \mathcal{L}(f(x_n), y_n) \quad \text{”Empirical risk”}.$$

Trying to make  $\mathcal{R}(f)$  small by minimizing  $\hat{\mathcal{R}}(f)$  is called **empirical risk minimization** (ERM). ERM is the main idea in this course.

## Risk



### **i** Question

Recall our QSAR application.

- ▶ Describe a use case where the IID assumption might be reasonable
- ▶ Describe a use case where the IID assumption might not be reasonable
- ▶ Does minimizing  $\mathcal{R}(f)$  guarantee an accurate prediction on any particular  $x$ ?

## Bibliography